Swifty.Devs

DOCUMENTO DE REQUERIMIENTOS - VERSION 3.0

Sistema de Timbre Escolar Inteligente

Aplicacion movil Flutter + Modulo fisico ESP32 Backend propio con FastAPI y PostgreSQL

Cliente	**I.E. Santa Sofia - Dosquebradas, Colombia**
Desarrollado por	**Swifty Devs**
Version	**3.0**
Fecha	**Junio 2026**
Plataforma	**Flutter - FastAPI - PostgreSQL - ESP32**
Cambios v3.0	**Arquitectura migrada a backend propio. Firebase eliminado completamente.**

CONTENIDO

Indice

01	Vision General	- - - - - - -
02	Arquitectura del Sistema	- - - - - - -
03	Requerimientos Funcionales	- - - - - - -
3.1	Gestion de Usuarios y Autenticacion	- - - - - - -
3.2	Modulo de Horarios	- - - - - - -
3.3	Timbre Manual	- - - - - - -
3.4	Modulo de Deteccion de Sismos	- - - - - - -
3.5	Pantalla OLED en Modulo Fisico	- - - - - - -
3.6	Operacion Offline	- - - - - - -
3.7	Modulo de Profesores y Notificaciones de Clase	- - - - - - -
04	Requerimientos No Funcionales	- - - - - - -
05	Stack Tecnologico	- - - - - - -
06	Plan de Desarrollo por Fases	- - - - - - -
07	Entregables Esperados	- - - - - - -

01 - VISION GENERAL

Vision General

Sistema integrado de control de timbre escolar compuesto por una **aplicacion movil Flutter** y un **modulo fisico basado en ESP32**. El backend es propio: **FastAPI + PostgreSQL** alojado en servidor dedicado, lo que garantiza control total sobre los datos, independencia de plataformas externas y escalabilidad a largo plazo. Las notificaciones push se entregan mediante FCM (Firebase Cloud Messaging) unicamente como canal de entrega, sin depender de ningun otro servicio de Firebase.

02 - ARQUITECTURA

Arquitectura del Sistema



03 - REQUERIMIENTOS FUNCIONALES

Requerimientos Funcionales

3.1 Gestion de Usuarios y Autenticacion

ID	Requerimiento
RF-01	El sistema contara con autenticacion propia mediante JWT (JSON Web Tokens) gestionada desde el backend FastAPI.
RF-02	Existira una cuenta de **Administrador Principal** (Dona Andrea) creada en la inicializacion del sistema, no eliminable.
RF-03	El administrador principal podra crear, editar, suspender y eliminar usuarios secundarios.
RF-04	El sistema manejara tres roles: **Administrador**, **Operador** y **Profesor**.
RF-05	Los operadores podran consultar horarios y activar el timbre manual, pero no modificar horarios ni gestionar usuarios.
RF-06	El administrador podra asignar y revocar permisos individuales por usuario.

3.2 Modulo de Horarios

ID	Requerimiento
RF-07	El sistema permitira definir un **horario habitual** semanal con multiples timbres por dia.
RF-08	Cada timbre tendra: hora, dias activos (L-S), etiqueta descriptiva (ej. Inicio de clases) y duracion del sonido.
RF-09	El administrador podra crear, editar y eliminar timbres del horario habitual.
RF-10	El sistema soportara **horarios especiales** que suspenden temporalmente el horario habitual.
RF-11	Al activar un horario especial, se habilitara un **date picker** para definir el rango de dias de vigencia.
RF-12	Al vencer el rango del horario especial, el sistema retomara automaticamente el horario habitual.
RF-13	Solo el administrador podra activar, modificar y desactivar horarios especiales.

3.3 Timbre Manual

ID	Requerimiento
RF-14	La app contara con un boton de timbre manual visible en la pantalla principal.
RF-15	El modulo ESP32 contara con un boton fisico de timbre manual.
RF-16	Ambos botones ejecutaran el timbre de forma inmediata, independientemente del horario activo.
RF-17	Cada activacion manual quedara registrada en PostgreSQL con usuario, timestamp y origen (app / fisico).

3.4 Modulo de Deteccion de Sismos

ID	Requerimiento
RF-18	El ESP32 contara con un acelerometro (MPU-6050) para deteccion de vibraciones sismicas.
RF-19	El algoritmo de deteccion debera **distinguir entre sismos y vibraciones de vehiculos**, usando analisis de frecuencia y duracion de la vibracion.
RF-20	Un evento sismico confirmado enviara una **notificacion push FCM** al dispositivo del administrador.
RF-21	La notificacion incluira: hora del evento, intensidad estimada (en g) y duracion de la vibracion.
RF-22	Al confirmarse un evento sismico, el modulo ESP32 **activara el timbre de forma inmediata y autonoma**. Simultaneamente FastAPI enviara la notificacion push al administrador como registro del evento.
RF-23	El historial de eventos sismicos detectados sera consultable desde la app.

3.5 Pantalla OLED en Modulo Fisico

ID	Requerimiento
RF-24	El modulo incluira una pantalla OLED de 1.3" (SH1106).
RF-25	La pantalla mostrara en tiempo real: hora actual, nombre del proximo timbre y tiempo restante.
RF-26	Al activarse el timbre, la pantalla mostrara un indicador visual de timbre activo.
RF-27	Si no hay conexion a internet, la pantalla indicara el modo offline con un icono visible.

3.6 Operacion Offline

ID	Requerimiento
RF-28	El ESP32 almacenara localmente el horario activo en memoria flash (SPIFFS / LittleFS).
RF-29	La app Flutter utilizara cache local para permitir consulta y edicion de horarios sin conexion.
RF-30	Los cambios realizados offline se sincronizaran automaticamente con el backend al recuperar conectividad.
RF-31	El modulo ESP32 operara el timbre automatico de forma autonoma aunque pierda conexion al servidor.

3.7 Modulo de Profesores y Notificaciones de Clase

NUEVO EN v2.0

ID	Requerimiento
RF-32	Cada profesor tendra un perfil en el sistema con: nombre, correo, materia(s) que dicta y token FCM de su dispositivo.
RF-33	El administrador podra crear, editar y eliminar perfiles de profesores desde la app.
RF-34	Cada profesor tendra un horario semanal asignado con entradas que incluyen: dia, hora de inicio, hora de fin, materia y salon.
RF-35	El administrador podra cargar y modificar el horario semanal de cada profesor desde la app.
RF-36	Al activarse el timbre (automatico, manual o sismico), FastAPI consultara que clase corresponde a continuacion para cada profesor segun su horario en PostgreSQL.
RF-37	Se enviara una notificacion push FCM personalizada a cada profesor con: hora de la proxima clase, materia y salon al que debe dirigirse.
RF-38	Si un profesor no tiene clase en el siguiente periodo, no recibira notificacion.
RF-39	Los profesores tendran acceso a la app con rol **Profesor**, diferente al Administrador y al Operador.
RF-40	La app en modo Profesor mostrara el horario semanal completo del profesor autenticado.
RF-41	La clase en curso se resaltara visualmente con un indicador de color y etiqueta **En curso**.
RF-42	La proxima clase se resaltara con un indicador secundario y etiqueta **Proxima**.
RF-43	El horario del profesor se actualizara en tiempo real si el administrador realiza cambios durante el dia.

04 - REQUERIMIENTOS NO FUNCIONALES

Requerimientos No Funcionales

ID	Requerimiento
RNF-01	La app usara los **colores institucionales** del colegio como paleta principal.
RNF-02	La interfaz seguira los lineamientos de **Material Design 3**.
RNF-03	La app debe ser compatible con **Android 8.0+** e **iOS 14+**.
RNF-04	El tiempo de respuesta entre comando de la app y activacion del timbre fisico no debe superar **2 segundos** con conexion estable.
RNF-05	El envio de notificaciones FCM a todos los profesores debe completarse en menos de **5 segundos** desde la activacion del timbre.
RNF-06	Toda comunicacion entre la app y el backend usara HTTPS con certificado SSL valido.
RNF-07	Las contrasenas se almacenaran en PostgreSQL con hash **bcrypt**. Nunca en texto plano.
RNF-08	La autenticacion se realizara mediante **JWT** con tiempo de expiracion configurable.
RNF-09	El codigo Flutter seguira arquitectura **Clean Architecture** con separacion por capas (data / domain / presentation).

05 - STACK TECNOLOGICO

Stack Tecnologico

App Movil

Framework	Flutter (Dart)
State Management	Riverpod / Bloc
Cache local	Hive / Isar
Autenticacion	JWT almacenado en secure storage
Notificaciones	flutter_local_notifications + FCM

Backend

Framework	FastAPI (Python)
Base de datos	PostgreSQL
ORM	SQLAlchemy / Tortoise ORM
Autenticacion	JWT con python-jose + bcrypt
Servidor	AlmaLinux - Nginx reverse proxy
Dominio	host.swiftydevs.dev

Notificaciones Push

Servicio	FCM - Firebase Cloud Messaging (solo canal de entrega)
Integracion	FastAPI llama directamente a la API HTTP de FCM
Dependencia Firebase	Ninguna - solo se usa la Server Key de FCM

Modulo Fisico

Microcontrolador	ESP32 DevKit v1 / v4
Sensor sismico	MPU-6050 (acelerometro + giroscopio)
Pantalla	OLED 1.3" I2C (SH1106)
Actuador	Rele de estado solido 3.3V
Comunicacion	HTTP REST hacia FastAPI via WiFi

06 - PLAN DE DESARROLLO

Plan de Desarrollo por Fases

Fase 1	**Base del proyecto** — Mes 1 Proyecto Flutter + FastAPI configurados - PostgreSQL con modelo de datos - JWT Auth funcionando - Los 3 roles operativos
Fase 2	**Modulo de timbre** — Mes 2 Sincronizacion ESP32 con FastAPI - Timbre automatico y manual - Operacion offline - Pantalla OLED funcionando
Fase 3 - 4	**Modulo de profesores** — Mes 3 - 4 CRUD de perfiles y horarios de profesores - Vista semanal con clase en curso - Notificaciones FCM desde FastAPI
Fase 5	**Modulos avanzados** — Mes 5 Horarios especiales con date picker - Modulo sismico con algoritmo anti-vehicular - Historial de eventos
Fase 6	**Pulido y entrega** — Mes 6 UI con colores institucionales - Pruebas reales con ESP32 fisico - Correccion de bugs - Documentacion final

07 - ENTREGABLES

Entregables Esperados

01	**App Flutter (Android + iOS)** Modulos de usuarios, horarios, timbre manual y vista de profesores.
02	**Backend FastAPI** API REST documentada con todos los endpoints del sistema.
03	**Base de datos PostgreSQL** Modelo relacional completo con migraciones y datos iniciales.
04	**Firmware ESP32** Codigo fuente documentado para el modulo fisico.
05	**Manual de usuario** Guia basica para el administrador (Dona Andrea).
06	**Diagrama de flujo** Algoritmo anti-vibracion vehicular para el modulo sismico.

Swifty.Devs - Built for Apple. Day 0.